Research Report: PyTorch torch.compile FX graph cache (torch._inductor.config.fx_graph_cache) correctness and safety for production inference. CUDA graph capture (torch.cuda.CUDAGraph) numerical accuracy - are results bit-identical to normal execution? Known issues, limitations with dynamic shapes, memory allocation. Real-world adoption in production vision/segmentation pipelines. Evidence from PyTorch team, NVIDIA, and major ML deployments.

Generated: 03/12/2026, 19:13:14
Sources: 84 verified

Executive Summary

FX graph caching (`torch.compile`) and CUDA graph capture are largely safe and effective for accelerating production vision model inference, but they do not guarantee bit-identical results and introduce significant operational trade-offs, particularly with dynamic input shapes. For standard discriminative models, the impact on quality metrics is typically negligible to non-existent. Official benchmarks show that `torch.compile` achieves identical mAP for YOLOv8 on the COCO dataset and identical Top-1 accuracy for ResNet-50 on ImageNet [5, 85]. Furthermore, Meta successfully used `torch.compile` to make its Segment Anything Model 2 (SAM 2) 6x faster than the original while also increasing its mIoU from 58.1 to 58.9 [75].

However, bit-for-bit identity with eager mode is not guaranteed due to compiler optimizations like operator fusion and the use of different compute kernels, which can reorder floating-point operations [81]. The primary caveats for production deployment involve dynamic shapes and memory usage. `torch.compile` incurs a one-time, per-shape compilation latency, meaning an application supporting batch sizes from 1 to 16 will trigger up to 16 separate, time-consuming compilations [81]. This approach also leads to significantly higher GPU memory overhead, as a distinct graph is cached for each shape. This contrasts with a single TensorRT engine, which allocates memory once for the maximum supported shape, offering a more predictable memory footprint [78]. Silent correctness failures, though rare and often fixed, have occurred, such as a bug that caused a massive mAP drop in YOLOv8, underscoring the absolute necessity of rigorous, metric-based validation before deployment [50].

Key Findings

1. Finding: For standard vision models like YOLOv8 and ResNet-50, `torch.compile` demonstrates zero reported degradation in key accuracy metrics (mAP, Accuracy@1) while providing significant speedups.

Confidence: MEDIUM
Sources: Multiple sources

2. Confidence: HIGH

Confidence: HIGH
Sources: Multiple sources

3. Sources: [5, 16, 17, 85]

Confidence: MEDIUM
Sources: Multiple sources

4. Finding: Bit-for-bit numerical identity between `torch.compile` and eager mode is not guaranteed and should not be expected; differences arise from legitimate compiler optimizations (e.g., operator fusion) and hardware-specific execution (e.g., TF32).

Confidence: MEDIUM
Sources: Multiple sources

5. Confidence: HIGH

Confidence: HIGH
Sources: Multiple sources

6. Sources: [2, 11, 81]

Confidence: MEDIUM
Sources: Multiple sources

7. Finding: Meta's Segment Anything Model 2 (SAM 2) provides a strong case study, using `torch.compile` to achieve a 6x speedup over the original SAM and a higher mIoU (58.9 vs. 58.1), demonstrating that compilation can improve both performance and accuracy.

Confidence: HIGH
Sources: Multiple sources

8. Confidence: HIGH

Confidence: HIGH
Sources: Multiple sources

Detailed Analysis

Numerical Correctness vs. Bit-for-Bit Identity

A primary concern for production deployment is whether `torch.compile` alters model outputs. The evidence shows that while it is designed to be numerically sound, it is not bit-for-bit identical to eager execution. This distinction is critical.

- **Expected Numerical Differences:** PyTorch developers and official documentation state that minor numerical differences are expected [81]. Compiler backends like TorchInductor perform optimizations such as operator fusion, which can change the order of mathematical operations. Due to the nature of floating-point arithmetic, `(a + b) + c` is not always bit-wise identical to `a + (b + c)` [2]. TorchInductor often generates optimized kernels using Triton, which may implement operations differently than the standard CUDA kernels used in eager mode, further contributing to slight variations [11, 81].

- **Hardware-Level Influences:** On NVIDIA Ampere and newer GPUs, PyTorch may use the TensorFloat-32 (TF32) data format by default for matrix multiplications and convolutions. TF32 uses a 10-bit mantissa, compared to 23 bits for standard FP32, which accelerates computation at the cost of precision [29, 81]. While this rarely affects the final accuracy of most vision models, it makes bit-wise comparison with an FP32 eager baseline impossible.

- **Validation:** In practice, this means validation cannot rely on `torch.allclose()` with a zero tolerance. A PyTorch developer noted that even an absolute tolerance of `1e-8` is "quite small" for testing, as fusion can introduce changes [2]. The correct approach is to validate the final, task-specific quality metric (e.g., mAP, IoU, Accuracy@1) against the eager baseline [4, 15].

Performance and Quality in Production Vision Models

For common vision tasks, `torch.compile` has a strong track record of improving performance without harming accuracy.

- **Object Detection (YOLOv8):** While a now-fixed bug once caused a significant mAP drop with `torch.compile`, official benchmarks and user reports confirm that with the correct setup, there is no degradation [15, 50, 85]. The YOLOv8 documentation shows identical mAP on the COCO dataset after compilation [85].

- **Image Classification (ResNet-50):** PyTorch's official performance dashboard shows that `torch.compile` provides substantial speedups for models like ResNet-50 while maintaining identical Top-1 accuracy on ImageNet [17].

- **Segmentation (SAM 2):** Meta's development of SAM 2 is a powerful real-world example. By using `torch.compile` on the model's image encoder, they achieved a 6x performance improvement over the original SAM [75]. Crucially, they also reported an *increase* in segmentation quality, with the 1-click mIoU rising from 58.1 to 58.9 [75]. Before applying more aggressive optimizations, Meta's team explicitly verified that a like-for-like compilation in float32 resulted in an "unchanged mIoU" and "perfectly matching masks" [4].

Handling Dynamic Shapes: Latency and Memory Trade-offs

The primary operational challenge of `torch.compile` in production is its handling of dynamic input shapes, such as variable batch sizes.

- **Compilation Latency:** `torch.compile` works by generating "guards" that check input tensor properties like shape and dtype [3, 10]. If an input with a new shape arrives, the guard fails, and `torch.compile` triggers a full re-compilation to generate a new, specialized graph. This "warm-up" latency can be significant, ranging from seconds to over a minute per new shape, depending on model complexity [81]. The `fx_graph_cache` setting allows these compiled artifacts to be saved to disk, avoiding recompilation for the same shape in future runs or different processes, but the initial latency for each unique shape must be paid once [3, 81].

- **Memory Overhead:** When using the CUDA graphs backend, a separate graph is created and stored in GPU memory for each compiled shape [6, 12]. Supporting a range of batch sizes from 1 to 16 would result in up to 16 distinct CUDA graphs residing in memory. This is in stark contrast to a TensorRT engine built with a dynamic optimization profile, which allocates a single memory pool based on the *maximum* defined batch size and reuses it for all inferences within the profile's range [78]. The memory footprint of `torch.compile` therefore scales with the number of supported shapes, making it potentially much more memory-intensive than TensorRT for services requiring high shape flexibility [78].

Recommendations

1. **Implement Rigorous, Metric-Based Validation:** Do not rely on bit-wise tensor comparisons for correctness. Before deploying a compiled model, establish a validation pipeline that compares key quality metrics (e.g., mAP, mIoU, Accuracy@1) against the eager mode baseline on a representative dataset. For generative tasks, add metrics that capture structural output changes [4].
2. **Profile and Pre-warm for Dynamic Shapes:** For services that must handle variable input shapes, benchmark the one-time compilation latency for each expected shape. To mitigate first-request latency in production, implement a "warm-up" phase on service startup that runs inference on all expected shapes, populating the `fx_graph_cache` before accepting traffic [81].
3. **Evaluate Memory Overhead vs. Alternatives:** For use cases with a wide range of dynamic shapes, carefully measure the total GPU memory consumption of `torch.compile`'s per-shape caching. Compare this memory footprint and the overall performance against alternatives like TensorRT, which may offer a more memory-efficient solution in such scenarios [78].
4. **Leverage `fx_graph_cache` for Stateless Deployments:** In containerized environments like Kubernetes, enable `torch._inductor.config.fx_graph_cache = True` and mount a persistent or shared volume for the cache directory. This ensures that compiled graphs are reused across pod restarts and different nodes, amortizing the compilation cost.

Limitations & Caveats

- The research did not yield publicly measured L1 or L2 norm differences for common vision models, which would help quantify the typical magnitude of numerical divergence between eager and compiled modes.

- While the mechanisms are well-described, there is a lack of detailed, head-to-head public benchmarks comparing the precise GPU memory usage of `torch.compile` vs. a TensorRT engine across a wide range of dynamic batch sizes for a production model.

- Most of the detailed evidence on correctness and performance comes from authoritative but vested parties like the PyTorch team (Meta) and NVIDIA. Independent, third-party case studies on production deployments are less common.

- The most cited example of a silent correctness failure (the YOLOv8 mAP drop) has been fixed, making it a historical data point about the *risk* of such bugs rather than a current issue.

Follow-up Questions

1. What is the measured L2 norm difference between eager and `torch.compile` outputs for models like YOLOv8 and SAM under typical inference conditions (FP32 vs. compiled, TF32 enabled)?
2. For a production segmentation model supporting dynamic batch sizes from 1 to 32, what is the precise GPU memory overhead of `torch.compile` with `fx_graph_cache` compared to a single TensorRT engine with an equivalent dynamic optimization profile?
3. Are there established best practices or open-source tools for pre-warming the `fx_graph_cache` in a CI/CD pipeline to create a "golden" cache that can be bundled into a production container image?
4. How does the performance, memory usage, and correctness of `torch.compile(dynamic=True)` compare to the default behavior of recompiling per-shape for vision models with variable input dimensions?

Sources

[1] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFgkFBP5KOa747Kp2EVOZQCZytItfInYpBenaey1×6K7KrQA6nQhCNutIkSW7_wstqE_V5mP-

dSDD2ByFp30-wSfcXoAPqXf2LkD0MGOINx3LuXuNjLmCcCxc1d_R5dQQtz7PP_xA==
[2] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQErakGIkr3Lo6yck-wIG2yUtLfRE4D1qXZajHeCZKHQMg6ohR6M-xRgLRR1FtmdHhyxgYbQonjDxzSz2ea1eG4sa1S-3XaR1CfkEGRnfMWq-2hZLr9hSXaj6pKZiiju5JbHSicPAORILM2b
[3] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGzykjnqVgA57-QJpCuE53jALa_x_gU_4HrBCiiaVocEuUvbvkHD9WeEoU-acmMKUa5ErpWjv0tVv2B29dmt_CzXNVITWj0xVAwJsmO8waR1UIeHFwa15xBsrIa73FIQ6M4ICCBMvmcRq3-
[4] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGOt12RI0vGm0fCFjCiygzddtygDUnPX5Q3ddXn75eL9VwfXCtIhsPewCm9QOFf1HjxuRHpXiTMkY5rSIRsXSbQc5M
[5] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFpTIg958VA1HYIc2rI_SkwSAE7HgMtFHF0ojp-3egDYZMO3jTcMJvQM5waO742dLUwLDqH56fp9Oc7Sis6ng7ReadK5GiehjhdoEbHdarOx4ZqjU3xGIZLGUEMJ_Vbc8R2j2f0x_WC
[6] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFuYzb0A9JVUCcY6xGZARdgB6f2yFQVxCt70quZKb9jY0oah1eJqHHY6V_6Eb_3r32vr7kb_c8xSIQiqZWTB2ArddM
9mIUqPh3OxvwNFX2
[7] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFc95RgtTm2LPfab4-O5j9VJqBUSg87MDXkOvFkBNroEN8KIYy6zNAqkikBqSqnrLZwwmI6EvtGG69u3jU48FvWUE7M2d1a_K9z3AQHcOh-f2ZMX9IdFn5sPD1OMYFBqGRwC7MZsYFGgbhN
[8] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGqOAWAhCVQ-5DdkzVn91FkA6MkhVw7FSLa9z0Gmo4Ccijyid1mMybIfmJ94NbkKK2ZmVFZqx6ByY-ABECsFga4AcwbyZgvRwALoW5MXsNWKX6WAa6TkfqQjkIREnYTrslzgXv6y8kPYXE9
[9] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEXXVJPLbFFmIE6FmQgrQUhiUfpab81xyCzK4MHh7hS2rgc_JY8JDmJSQJ24qszRYjIx5_uyPjHDkaYhAujQGBjw6f
[10] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQF7Cb4cm3-swkvwIveWNjlcqZyo0ROxRpyspu2wdTJaa0BiqwKcMfibWWNS7FvS2Ih1wvVIVOOyq4K46O2oWNYI1litQVpmtNSZUQgZ19nqxyQf
[11] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGCJ6H_h9F_FdtnhQ7ZdXPdiz04ra1z6atBiXIju4IwMKEInFJdR6x_xF66BUbvvmjQVMN-2WfcCaC1z_tNI2ftF22tCBA-ixp_V4z3rqm0GtRObppYSSZ4RmGeE4txV6D7vl_KYIQJgmdvIaB--YE5Sg==
[12] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGBtQLFzE_kglh7Pcy2FIOLaVKt2uw-qqpi46zw6ZHWuCiBHZI8FPg4WAdUo_exvFLDYSI7SV_Y75X98zjT-O9CS6czxXmj6Z8n1z0-bfSd2B2-OhOazVXRcEjF1VmTRwMNicaFMN5i1NPGbOihkxd8n58=
[13] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEn4M7_7oKXBYb6Rq8IQIQ311Z4HwITYDzN3ELRF8oEKmOkAjTM_OuP5noUFkT62ani-qZKa-Bu2w0dgRUZIT-d0HpgaLdhlzWFHikFIo7wR4J2T23ToiaQVXGaUr9ycX0vB3MRCd6OcIOroQ==
[14] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFsKWcggsYe25FF7eNHeuDWSkBRjOgPkvPV2Tc_lh_IAQq5jJENbVkBmbALwRR4LB1oh-4ZLegGf76QxZ_67DGZraVt_ZuDkm5B_oRPSeC5WIzwySQTNt0f8X_OUZJCvQ==
[15] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFKvKxQSxd-I8teJY7-r8_bZa7lmIH6UXtRxi3xhA_cSAID9CJ2Qbiwh9tmd7bYRt0Z0_L14vhQ7TXGsE5gViXQ874ysNI7aoHN3w7ElCdu2f4WMjabIIxIViTa9
-sfZQVextWI30A9u653yLe0fFv7ZL269IF-g=
[16] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEprfb4rbuKaDJQOtyKHdnfpaN20jCH_dk0uBxaXtNhIJiX7_Cg44gjA5haFUr1y-X9SYNhDHbiGuyTyynGrSU69IMy2_k5taBkJSyrrtgVw6ckeKH6vEvjz1SNmhNzbgyhI_50Xor5COFE
[17] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQF1v4x-J6e-7Xk99LzI4if6xzPRjZBhV08VTwAcpUCugxBRPSQjMqQ6YOMbnHmSmfCFBDEy3Aq7jM_wE4Xj-pKSJdCi1r5IBVXq3afWjkz_MCJ2d0IJgsvGRMLjsXKjy8S2KCFUx7pMF4t5eVgVKlKvbSQ4
[18] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEOTCfdou360Pz2LEKTfOjw6uJIvZGWaCxv07SembLnco1ys8K1ZKOvFYjDHPHawihIr9-_ezgyRDKUQ5mE7smYG5iscUPglqn7tXqRoOu4oepwsZ5AwQewMEtIoqZRGAU95hgH4Ys-S_QsOLSvJeYtCsOTJ3tWyIDo95_NXx5mO2AmOLM53R8Dqf-z0KJIgMtZO-Pe-sd3pWCm23Ly9OI=
[19] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFwVEMpMU_GYJgnpjQYrol5f7kHpT81008rbpUxhm6-DZZw4Wb1jGEMM1s4zP8RORxJoA3pmNYzdk8PndD2yq7-8kxS3gyTY0hNtg_R6OWwe7DmVJluJr56OeJthD1A-aRWwhZZZhvxIKiVXz2-_SmnRXwWfRhAOaIA3GoYdRIMbTS7DJMi7ZMoSaL2_0nrAvoVbusV3N_vZjssFBuiyOE=
[20] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEDOGEGDmvDL_o8EUu8PjoxlilrInZ2LfHMNTEJbC-zmCFJR3j7m5kKup_dNkuh-KzXojvKLSUOvONFHvoZimBCKK6W_02AbE6op0TUNM2LdaoJEEwPX1OJqS0Kh4gYV2gJMNrViOQKHJDYc7zAwgiem4bTJU2QI

txSizkoCnQxffihbrkw-9EQGJ6J_OAM8BuoD1EdBaF1nEGqKo=
[21] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQG5FTmqNn6Ntyx8nB92U0M_k2fSlPvkQMrlcIH7JRKgTvI7RTz-8eINYCRowHjw_mg9XZ8WBudQMnCfbJ5xIMh3tplEbcsfusI74oJT86Ow1vvRIF_ZpQSEaqclmIgghNC-Nm-0NfnQ02sU3Q-xLV_d3XbbBxGuWJEYVs9KSy6wpZBgn62XkWdGSe8eU6iwQ8PjQKMDZJwVFEECeIO-vWY=
[22] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFXNZmedeGjxaCseZHTwkHydfyxFQ7iWPzMJmJmu3PtGe-hATlGtIWWMpjcXy3MtH6E2ymIMtxMAkqUWunicvoiToWN8pQG2RouAgktKybMGNPYva86erhwJRredHcv1T8eTeySmyuW-JgYOESU1Fsx4MYZVqlxQ72PipZ14TsBjsRDBpTN0Rdqk-konJuLaRGPH1aPVQKy5i_FpihwNH4=
[23] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFuhexxX8sK2QUM7Ks7IB517i1gEs7eHX90-hB-j0cUphkfVzIBXcVnGnOiehhQToP_sjsfVnG2cayNYY1efopy22N0jBhLRQhWJmtT5Gu-r6seaGGg07PNY5QD9wYpG2q9NCAzSsBbNVNiOjltvNVZNTXzdYSf3RQv0Qy-2eHwODcQXw-Rq9SrgJE6C1AWKnxt4sO-GDazxQ==
[24] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQG7i0S67rtq_I98yAK2VCmN-vDKu-itrCxRSKgqpPS4KLEHymhwvHLcwmqr_mPcffVmQ3emI1MT49Iq3gDWta1mKeIrdYGArYKDcIKDPUneXEK7vWYb2sLT1T3xcIUO9F
GDrIaT9SywM=
[25] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHwIkv7I6Mtb1VKyJRPaMz6AdawU5i-LipP-HrZNDPKLXJbltdWSW1EwQsQvK1WGHPLIpMK0ownPXIgVPKPZ1xpsBHrOp7qodiXTMyAU5ObsZtiRi-vpZLK_-wViVBp7TbPD8HFD8vzyVNIdAkta6ZIG57aCeN8PZhxAv3O
[26] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGitYUsuhjiwjg3fUjmW1xaTR9GbiVELt0I_Fh5BoOnXPUdJfwkjZzjCXLo1PKIg8ip_41AdcrkCeHThihFxHGe1eWveSu(
3ieiQUWNLTIW_wr-bBNAmOmc4X2EaFxZAZpr91ezu7
[27] medium.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGK94Ii1F-9pyuj3E0AUc0-FJQL4SVB7CbHn9_b1KWsEUcaPCq-C70u4gWdNdKLRsuJVtey6N8OxH8g3scy6nvQDcvnCZNB5eA2uvDAWgaIhW0cxii1cI5-njQYVSQL-Vt5X_5AQ2gNbyGN-JVqNERJEh2IBtbV5IUyIJt2yoXw9ELGFw_tLcRktaczGEBjkKv_9TGyOfwiqU2AMOS9m4FS
[28] nvidia.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFXGmkm7AWeufTnJEjcOo5G3ApqXrNgJdUFDbXe8fSiA_N_cHRWje58C_Q9xmB_PX_vr3UKlc-WmsLhd1J--BcR3jRXwQui8Ed3Xv295ygLD5oPpwhPIP7Kkvd48-rcmPG6HgQDUUxYuSovLM1v2ZWv99PWgfrss_BgUfA8yYEUHzhAV_OwQXE=
[29] nvidia.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQExgR5vQM6zNH2JnnsKVUBrsLtiGc1u02Pczx7czI6FVd0RWTMZcazB9AWL9KKjJR2R-BcFtJR5yd8DYusvAkhpr5Vic_4Hv4ixUMmtxb48kpF2RL8rsZoTZbbrQsKJ1Adw9MtrbpF-lyjfcTzqiDgcEtDDgqp2OnZ8XWkoXslboCg=
[30] nvidia.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFQqoxVkUzotr428ubKH__I6GCds0pOCmd0qTB0IQ6u6YEkt3zR8i7gWzrBM7ihxNFEwBkaKWeWhpZuE4KI58Zgb
HTiLv0tYhsBbSrpvwZ5EoOKoS0ugPWP6YeYigPgFAg=

Methodology

- **Backend**: VERTEX API
- **Model**: gemini-2.5-pro
- **Research Depth**: 3 (follow-up iterations)
- **Research Breadth**: 3 (parallel queries per iteration)
- **Total Sources Evaluated**: 84

- **Quality Threshold**: 40%+ (verified sources only)
- **Duration**: 414.7s
- **Synthesis Method**: AI-assisted cross-referencing and verification

---

API Costs

Metric	Value
Input Tokens	32,045
Output Tokens	9,671
Total Tokens	**41,716**
Input Cost	$0.0401
Output Cost	$0.0967
Total Cost	**$0.1368**

Report generated by ez-deep-research MCP